

Personal HTML/CSS Tutorial

This tutorial explains the current HTML and CSS files of the BFME II Archive project.

It only covers HTML and CSS. JavaScript is not used yet and will be explained later in the semester.

1. Overall structure of the website

The website currently consists of five HTML pages and two CSS files.

HTML pages

- `index.html` — the fullscreen homepage with four navigation links
- `maps.html` — the map archive page with example map entries
- `other.html` — the page for non-map downloads, currently the RotWK Trainer
- `about.html` — the text page explaining the project
- `imprint.html` — the legal/imprint page

CSS files

- `index.css` — styling only for the homepage
- `pages.css` — shared styling for `maps.html`, `other.html`, `about.html`, and `imprint.html`

This separation exists because the homepage has a different visual concept from the other pages. The homepage imitates a game menu, while the content pages use a simpler oldschool archive style.

2. `index.html`

`index.html` is the start page of the website. It is the page that visitors see first when they open the project.

```
<!DOCTYPE html>
```

This tells the browser that the document uses modern HTML5. It should always be the first line in an HTML file.

```
<html lang="en">
```

The `<html>` element wraps the entire page. `lang="en"` says that the main language of the page is English. This helps browsers, screen readers, and search engines understand the page.

```
<head>
```

The `<head>` contains information about the page that is mostly not visible directly on the page.

```
<meta charset="UTF-8">
```

This defines the character encoding. `UTF-8` makes sure that normal text and special characters display correctly.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

This helps the page scale correctly on different screen sizes. It tells the browser to use the actual device width and a normal zoom level.

```
<title>BFME II Archive</title>
```

This sets the text shown in the browser tab.

```
<link rel="stylesheet" href="index.css">
```

This connects the homepage to `index.css`. Without this line, the browser would only show unstyled HTML.

```
<body>
```

The `<body>` contains everything that is visible to the user.

```
<main>
```

The `<main>` element contains the main content of this page. On the homepage, the main content is the menu navigation.

```
<nav aria-label="Main navigation">
```

The `<nav>` element marks a navigation area. `aria-label="Main navigation"` gives the navigation a clear name for screen readers.

```
<ul>
```

The navigation links are placed in an unordered list. This is a common and semantic way to structure a menu.

```
<li><a href="maps.html">Maps</a></li>
<li><a href="other.html">Other</a></li>
<li><a href="about.html">About</a></li>
<li><a href="imprint.html">Imprint</a></li>
```

Each `<li>` is one list item. Each `<a>` is a link. The `href` attribute defines where the link goes.

- `maps.html` opens the map archive
- `other.html` opens the non-map downloads page
- `about.html` opens the project information page
- `imprint.html` opens the imprint page

The homepage does not include a link to itself, because it is already the entry point.

3. `maps.html`

`maps.html` is the main functional page of the project. It shows a list of maps.

The beginning of the file is similar to `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
```

These elements again define the file as an HTML5 document, set the language to English, and open the metadata section.

```
<link rel="stylesheet" href="pages.css">
```

This connects the page to `pages.css`, not `index.css`. This is important because `maps.html` should use the oldschool archive style, not the fullscreen menu style.

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Maps | BFME II Archive</title>
```

These lines define character encoding, viewport behavior, and the browser tab title.

```
<body class="maps-page">
```

The `<body>` contains the visible content. The class `maps-page` is important because the CSS uses it to decide that the map entries should use the five-column layout. Without this class, the map sections would stack vertically like on the simpler content pages.

```
<header>
```

The `<header>` contains the top part of the page: the page title and main navigation.

```
<h1>Maps</h1>
```

The `<h1>` is the main heading of the page. It tells the user that this is the Maps page.

```
<nav aria-label="Main navigation">
```

This navigation appears on the content pages so users can move between all pages.

```
<ul>
  <li><a href="index.html">Home</a></li>
  <li><a href="maps.html">Maps</a></li>
  <li><a href="other.html">Other</a></li>
  <li><a href="about.html">About</a></li>
  <li><a href="imprint.html">Imprint</a></li>
</ul>
```

This list contains all main page links. Unlike the homepage, the content pages also include `Home`, because users need a way back to the start page.

```
<main>
```

The `<main>` element contains the central content of the Maps page.

```
<section aria-labelledby="map-list-heading">
```

This `<section>` groups the entire map list. `aria-labelledby="map-list-heading"` connects the section to its heading.

```
<h2 id="map-list-heading">Map List</h2>
```

This is the heading for the map list section. The `id` is used by `aria-labelledby`.

Map entries

Each map entry is an `<article>`.

```
<article>
```

An `<article>` is used because each map is its own independent content block. A map entry has its own title, images, metadata, description, and download link.

```
<h3>Harad Coast</h3>
```

The `<h3>` is the title of one map entry. The page has one `<h1>`, then the map list has an `<h2>`, and each individual map has an `<h3>`.

```
<div>
```

The `<div>` groups the five blocks of one map entry. It has no special meaning by itself, but it is useful for layout. In `pages.css`, `article > div` becomes a grid with five columns.

Minimap block

```
<section aria-label="Minimap">
  <h4>Minimap</h4>
  
</section>
```

This section contains the minimap preview. It uses `<section>` because it is one clear part of the map entry.

The `<h4>` labels the block visually as “Minimap”.

The `<img>` displays the minimap image.

- `src` is the path to the image file
- `alt` describes the image if it cannot be loaded and helps screen readers

Hard facts block

```
<section aria-label="Hard facts">
  <h4>Hard Facts</h4>
  <dl>
    <dt>Player</dt>
    <dd>8</dd>
```

```
...
</dl>
</section>
```

This section contains compact structured data about the map.

The `<dl>` element is a description list. It is used because the content is made of label-value pairs.

- `<dt>` is the term or label, for example `Player`
- `<dd>` is the value, for example `8`

This fits the map data well because entries like `Neutral Buildings → 3x Inn, 2x Outpost` are not normal paragraphs but structured facts.

Description block

```
<section aria-label="Description">
  <h4>Description</h4>
  <p>...</p>
  <dl>
    <dt>Creator</dt>
    <dd>Unknown</dd>
    <dt>Year</dt>
    <dd>Unknown</dd>
  </dl>
</section>
```

This section contains the longer map description and then the creator/year information.

The `<p>` element is used for normal paragraph text.

Creator and Year are again stored in a `<dl>` because they are label-value pairs.

Screenshots block

```
<section aria-label="Screenshots">
  <h4>Screenshots</h4>
  
  
</section>
```

This section contains additional screenshots. These are separate from the minimap. The minimap shows the map overview, while the screenshots preview the visual appearance of the map.

Download block

```
<section aria-label="Download">
  <h4>Download</h4>
  <a href="data/downloads/maps/harad-coast.zip">Download ZIP</a>
</section>
```

This section contains the download link. The `<a>` element links to the ZIP file for the map.

The text “Download ZIP” makes clear what kind of file the user receives.

Repeated map structure

The same `<article>` structure is repeated for the example maps:

- Harad Coast
- Helm's Deep
- Fords of Isen

This creates a real list and allows the CSS layout to be tested with multiple entries.

4. `other.html`

`other.html` contains non-map downloads.

```
<link rel="stylesheet" href="pages.css">
```

This page also uses the shared oldschool content-page styling.

```
<h1>Other</h1>
```

The main heading identifies the page.

```
<nav aria-label="Main navigation">
```

The same navigation structure is used as on the other content pages. This keeps the website consistent.

```
<section aria-labelledby="other-downloads-heading">
```

This section groups the non-map downloads.

```
<h2 id="other-downloads-heading">Other Downloads</h2>
```

This heading labels the section. The `id` is referenced by `aria-labelledby`.

```
<article>
```

The RotWK Trainer is placed in an `<article>` because it is one independent downloadable item.

```
<h3>RotWK Trainer</h3>
```

This is the title of the download entry.

```
<p>A small trainer tool for The Rise of the Witch-king.</p>
```

This paragraph briefly describes the file.

```
<dl>
  <dt>File type</dt>
  <dd>RAR archive</dd>
</dl>
```

This describes the file type as a label-value pair.

```
<a href="data/downloads/other/rotwk-trainer.rar">Download RAR</a>
```

This is the download link for the trainer archive.

5. about.html

`about.html` explains the purpose of the project.

It uses the same basic page structure as `other.html` and `imprint.html`:

- `<!DOCTYPE html>`
- `<html lang="en">`
- `<head>` with metadata
- `<link rel="stylesheet" href="pages.css">`
- `<body>` with header, navigation, and main content

```
<h1>About</h1>
```


This identifies the page.

```
<section aria-labelledby="about-heading">
```

This groups the About content.

```
<h2 id="about-heading">About This Project</h2>
```

This is the heading for the main text section.

```
<p>...</p>
```

The About page uses several paragraphs. This is correct because it is mainly a readable text page, not a list of structured files.

The paragraphs explain:

- that the website is a small archive for BFME II / RotWK files
- that maps are the main focus
- that additional files are placed in the Other section
- that the project is a fan project and part of a university seminar

6. imprint.html

`imprint.html` contains the imprint/legal information.

It uses the same structure as the other content pages.

```
<h1>Imprint</h1>
```

This identifies the page.

```
<section aria-labelledby="imprint-heading">
```

This groups the imprint content.

```
<h2 id="imprint-heading">Imprint</h2>
```

This is the section heading.

```
<p>Name: Your Name</p>
<p>Address: Your Address</p>
<p>Email: your@email.com</p>
```

These paragraphs are placeholders for the required imprint information.

```
<p>This website is a private fan project created for educational purposes as
part of a university seminar.</p>
```

This paragraph explains the context of the website.

7. index.css

index.css controls only the homepage.

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

The `*` selector targets every element. This removes default browser margins and padding. `box-sizing: border-box` makes element sizing easier because padding and borders are included in the total width and height.

```
html,
body {
  width: 100%;
  height: 100%;
}
```

This makes both the document and the body fill the full browser window. This is necessary for the fullscreen homepage.

```
body {
  overflow: hidden;
  font-family: Georgia, "Times New Roman", serif;
  background-image: url("data/images/homepage-bg.jpg");
  background-size: cover;
  background-position: center;
  background-repeat: no-repeat;
}
```

```
color: #e8d6a0;
}
```

This styles the homepage body.

- `overflow: hidden` prevents scrolling and visible scrollbars
- `font-family` uses a serif font to feel more old/fantasy-like than Arial
- `background-image` loads the homepage background image
- `background-size: cover` makes the image fill the screen
- `background-position: center` keeps the image centered
- `background-repeat: no-repeat` prevents tiling
- `color` sets the default text color to a warm gold tone

```
main {
  width: 100%;
  height: 100%;
  position: relative;
}
```

The `<main>` element fills the full screen. `position: relative` makes it possible to position child elements absolutely inside it, especially the navigation.

```
nav {
  position: absolute;
  left: 50%;
  bottom: 55px;
  transform: translateX(-50%);
}
```

This positions the navigation near the bottom center of the screen.

- `position: absolute` removes the nav from normal document flow
- `left: 50%` places its left edge at the horizontal center
- `transform: translateX(-50%)` shifts it back by half its own width, so it is truly centered
- `bottom: 55px` places it above the bottom edge

```
nav ul {
  list-style: none;
  display: flex;
  gap: 18px;
}
```

This styles the navigation list.

- `list-style: none` removes bullet points
- `display: flex` places the list items next to each other
- `gap: 18px` adds space between the buttons

```

nav a {
  display: block;
  min-width: 155px;
  padding: 10px 18px;
  text-align: center;
  text-decoration: none;
  text-transform: uppercase;
  letter-spacing: 1px;
  color: #f0d894;
  background: linear-gradient(#3a2a16, #120d08);
  border: 2px solid #8f6b2f;
  box-shadow:
    inset 0 0 8px #000,
    0 0 8px #000;
  font-size: 18px;
}

```

This styles the homepage navigation links as menu buttons.

- `display: block` lets the link behave like a rectangular button
- `min-width` gives every button a consistent width
- `padding` creates space inside the button
- `text-align: center` centers the text
- `text-decoration: none` removes the default underline
- `text-transform: uppercase` makes the labels look more like menu buttons
- `letter-spacing` spreads the letters slightly
- `color` sets the text color to gold
- `background: linear-gradient(...)` creates a dark brown gradient
- `border` creates a gold/brown outline
- `box-shadow` adds inner darkness and outer shadow
- `font-size` sets the button text size

This is a RotWK-inspired approximation, not a perfect copy of the original game UI.

```

nav a:hover {
  color: #ffffff;
  border-color: #d6aa4f;
  background: linear-gradient(#5a3a18, #1a1008);
}

```

This changes the button when the mouse hovers over it.

- text becomes white
- border becomes brighter
- background becomes slightly warmer/brighter

This gives the menu basic interaction feedback.

8. pages.css

pages.css controls the shared oldschool style of the content pages.

```
/* pages.css */
/* Oldschool content-page style for maps.html, other.html, about.html,
imprint.html */
```

These comments explain the purpose of the file. Comments are ignored by the browser but useful for humans.

```
* {
  box-sizing: border-box;
}
```

This applies `border-box` sizing to all elements. It makes layouts easier because padding and borders are included in width calculations.

```
body {
  margin: 0;
  padding: 24px;
  font-family: Arial, Helvetica, sans-serif;
  background: #111;
  color: #ddd;
}
```

This sets the basic page look.

- `margin: 0` removes the browser's default outside spacing
- `padding: 24px` adds space inside the browser window
- `font-family` uses a practical sans-serif font for readability
- `background: #111` creates a very dark background
- `color: #ddd` makes the text light gray

```
header {
  border: 2px solid #777;
  background: #222;
  padding: 16px;
  margin-bottom: 24px;
}
```

This styles the top header area.

- `border` gives it an oldschool framed look
- `background` separates it from the page background

- padding creates internal spacing
- margin-bottom separates it from the main content

```
h1 {
  margin: 0 0 12px 0;
  font-size: 36px;
  color: #f0d080;
}
```

This styles the main page heading.

- top/side margins are removed
- bottom margin creates space before navigation
- font size makes it prominent
- gold color gives it a game/archive feel

```
nav ul {
  margin: 0;
  padding: 0;
  list-style: none;
}
```

This removes default list spacing and bullet points from the navigation menu.

```
nav li {
  display: inline-block;
  margin-right: 12px;
}
```

This places the navigation list items next to each other instead of vertically.

- `inline-block` allows items to sit in one row while still behaving like boxes
- `margin-right` creates space between links

```
nav a {
  color: #f0d080;
  text-decoration: none;
  border: 1px solid #777;
  background: #111;
  padding: 6px 12px;
}
```

This styles the navigation links as small oldschool buttons.

- gold text
- no underline
- thin border

- dark background
- padding for click area

```
nav a:hover {
  color: #fff;
  background: #333;
}
```

This changes navigation links on hover. It gives feedback that the links are clickable.

```
main {
  max-width: 1500px;
}
```

This prevents the content from becoming too wide on very large screens. The maps page still has enough width for its five-column layout.

```
h2 {
  color: #f0d080;
  border-bottom: 1px solid #777;
  padding-bottom: 8px;
  margin-bottom: 20px;
}
```

This styles section headings.

- gold color connects them visually to the page title
- bottom border creates an oldschool divider line
- padding-bottom gives space between text and line
- margin-bottom separates the heading from following content

```
article {
  border: 2px solid #777;
  background: #1b1b1b;
  margin-bottom: 24px;
  padding: 14px;
}
```

This styles each map or download entry as a boxed item.

- border makes each entry clearly separate
- background creates contrast
- margin-bottom separates entries
- padding gives internal spacing

```
article h3 {
  margin: 0 0 12px 0;
```

```

color: #fff;
font-size: 26px;
border-bottom: 1px solid #555;
padding-bottom: 6px;
}

```

This styles the title inside each article.

- white color makes it stand out
- larger font size makes map names clear
- border-bottom separates the title from the content below

```

article > div {
  display: grid;
  grid-template-columns: 180px 260px 1fr 260px 160px;
  gap: 14px;
  align-items: start;
}

```

This is the key rule for the map layout.

It targets a `<div>` that is a direct child of an `<article>`. In `maps.html`, that div contains the five map blocks.

- `display: grid` activates CSS Grid
- `grid-template-columns` creates five columns:
 - `180px` for the minimap
 - `260px` for hard facts
 - `1fr` for the description, taking remaining space
 - `260px` for screenshots
 - `160px` for download
- `gap: 14px` adds spacing between columns
- `align-items: start` aligns all blocks at the top

This rule creates the intended layout:

Minimap | Hard Facts | Description | Screenshots | Download

```

article section {
  border: 1px solid #555;
  background: #111;
  padding: 10px;
  min-height: 100%;
}

```


This styles each inner block inside an article.

- border separates the blocks
- dark background gives them a boxed oldschool look
- padding creates internal spacing
- `min-height: 100%` helps the sections visually fill the article height

```
article h4 {  
  margin: 0 0 8px 0;  
  color: #f0d080;  
  font-size: 16px;  
}
```

This styles the small headings inside each map block, such as “Minimap”, “Hard Facts”, and “Download”.

```
img {  
  max-width: 100%;  
  height: auto;  
  display: block;  
  border: 1px solid #777;  
  background: #000;  
  margin-bottom: 8px;  
}
```

This styles all images.

- `max-width: 100%` prevents images from overflowing their container
- `height: auto` keeps the correct proportions
- `display: block` removes awkward inline image spacing
- border makes images look framed
- black background is useful if an image has transparent areas or fails to load
- margin-bottom separates stacked screenshots

```
dl {  
  margin: 0;  
}
```

This removes default margin from description lists so they align cleanly inside the boxes.

```
dt {  
  color: #f0d080;  
  font-weight: bold;  
  margin-top: 8px;  
}
```

This styles the labels in description lists.

- gold color makes labels easy to find
- bold text separates labels from values
- top margin spaces out each label-value pair

```
dt:first-child {  
  margin-top: 0;  
}
```

This removes the top margin from the first label in a list, so the list starts neatly at the top.

```
dd {  
  margin: 0 0 8px 0;  
}
```

This styles the values in description lists.

- removes default indentation
- adds space below each value

```
p {  
  margin-top: 0;  
}
```

This removes top margin from paragraphs so text starts cleanly inside boxes.

```
section[aria-label="Download"] a,  
article > a {  
  display: block;  
  text-align: center;  
  color: #111;  
  background: #f0d080;  
  border: 2px solid #fff0aa;  
  padding: 10px;  
  font-weight: bold;  
  text-decoration: none;  
}
```

This styles download links as buttons.

The selector has two parts:

- `section[aria-label="Download"] a` targets links inside download sections on the Maps page
- `article > a` targets direct download links inside articles, such as on the Other page

The rule makes links look like buttons:

- block display
- centered text
- dark text on gold background
- bright border
- padding for click area
- bold text
- no underline

```
section[aria-label="Download"] a:hover,  
article > a:hover {  
  background: #fff0aa;  
}
```

This changes download buttons when hovered. The brighter background gives visual feedback.

```
body:not(.maps-page) article > div {  
  display: block;  
}
```

This is a safety rule for pages that are not the Maps page.

The Maps page uses `<body class="maps-page">`, so it keeps the five-column grid.

Other pages do not have that class. On those pages, direct article divs should not accidentally become grid layouts. This rule makes sure non-map article layouts stay simple.

9. No JavaScript yet

The current project does not use JavaScript yet.

Later in the semester, JavaScript may be used for:

- loading real map data from `maps.json`
- generating map entries automatically
- searching maps by name
- filtering maps by player count
- possibly controlling sound on the homepage

These parts will be explained later in the semester.

10. How to explain the project in a presentation

A short explanation could be:

“This website is currently built with separate HTML pages and two CSS files. The homepage uses `index.css` because it has a special fullscreen game-menu design. The content pages use `pages.css` because they share a simpler oldschool archive style. The Maps page uses semantic HTML: each map is an `article`, and its structured facts are written as description lists. CSS Grid is used to place the five map blocks next to each other: minimap, hard facts, description, screenshots, and download.”